Code Splitting, Lazy Loading & Suspense Deep Dive

5 min read Updated 16 Jul 2026 Topic 26 of 62

5.2 - Code Splitting, Lazy Loading & Suspense Deep Dive

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"How does code splitting improve performance?"	🟡🔴	★★★★☆
"What is dynamic import?"	🟡	★★★☆☆
"How do you analyze your bundle size?"	🔴	★★★☆☆
"What is tree shaking?"	🟡	★★★☆☆

How Bundling Works

one or a few output files. For a small app, this single bundle might be 100KB. For a large app with heavy libraries, it can balloon to several megabytes.

The user has to download this entire bundle before anything appears on screen. Code splitting breaks that single bundle into multiple smaller chunks that load on demand.

Dynamic Import: The Foundation

The standard `import` statement is static. It loads the module at build time and includes it in the main bundle.

JSX Copy

```jsx
// Static import: always included in the main bundle
import { Chart } from 'chart.js';
```

A dynamic `import()` is a function call that returns a Promise. The bundler sees it and creates a separate chunk that only loads when the import is actually executed.

JSX Copy

```jsx
// Dynamic import: creates a separate chunk, loaded on demand
const module = await import('chart.js');
const Chart = module.Chart;
```

React.lazy uses dynamic import under the hood. But you can also use dynamic import directly in event handlers or useEffect to load heavy libraries only when needed.

JSX Copy

```jsx
function ChartPage() {
  const [ChartComponent, setChartComponent] = useState(null);

  const loadChart = async () => {
```

```
  return <button onClick={loadChart}>Show Chart</button>;
}
```

Bundle Analysis

You can't optimize what you can't see. Bundle analyzers visualize what's inside your bundle and how much space each dependency takes.

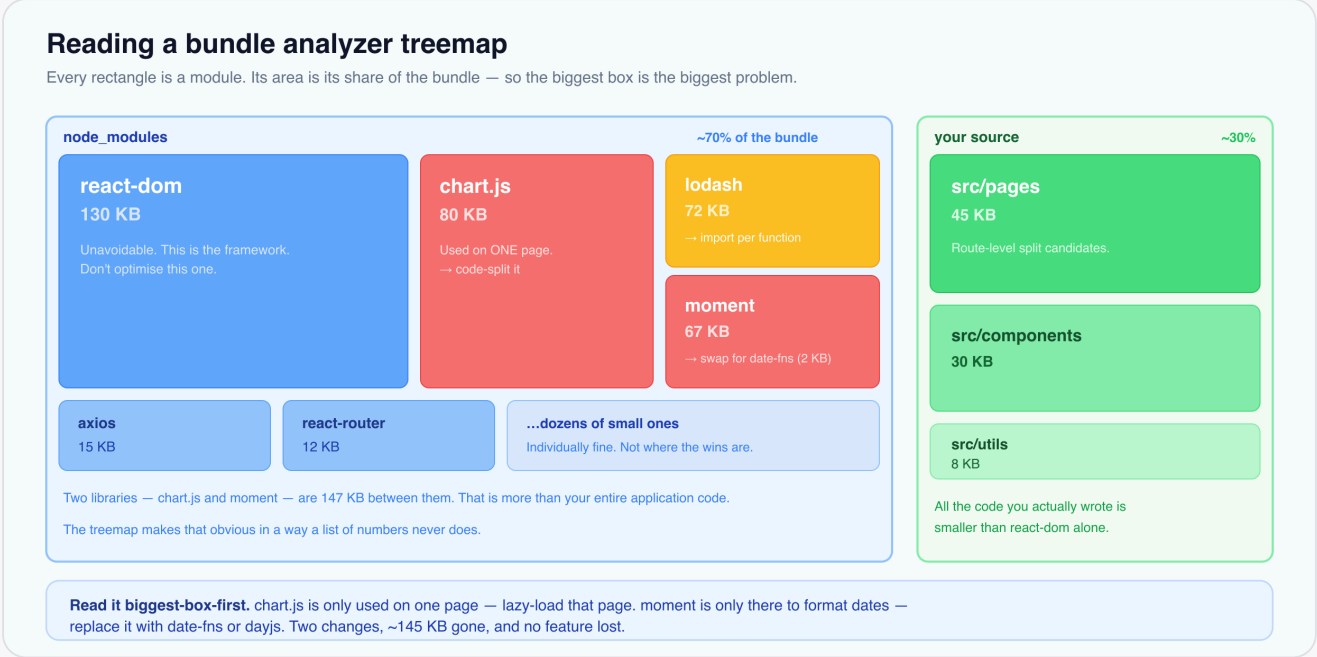
For **Webpack**: use `webpack-bundle-analyzer`. It generates an interactive treemap showing every module and its size.

For **Vite**: use `rollup-plugin-visualizer`.

BASH Copy

```
# Webpack
npm install --save-dev webpack-bundle-analyzer

# Vite
npm install --save-dev rollup-plugin-visualizer
```



Common findings and fixes:

Finding	Fix
Large library used on one page	Lazy load that page
Entire lodash imported	Use `lodash/specific-function` instead of full import
moment.js (67KB+)	Replace with date-fns or dayjs (2KB)
Duplicate dependencies	Check for version conflicts in package-lock.json
Unused exports	Enable tree shaking (use ES modules, not CommonJS)

Tree Shaking

Tree shaking is a technique where the bundler removes code that is imported but never actually used. It works automatically with ES module syntax (`import` / `export`).

JSX Copy

```jsx
// ✅ Tree-shakeable: bundler can remove unused exports
import { debounce } from 'lodash-es';
// Only debounce is included in the bundle

// ❌ Not tree-shakeable: pulls in the entire library
const _ = require('lodash');
_.debounce(fn, 300);
// Entire lodash (72KB) goes into the bundle
```

For tree shaking to work:

1. Use ES module syntax (import/export), not CommonJS (require/module.exports)
2. Use libraries that ship ES modules (lodash-es instead of lodash)
3. Avoid side effects in modules (or mark them in package.json)

Suspense Boundaries: Strategic Placement

From Article 3.3, you know Suspense provides a fallback while lazy components load. The strategy is about *where* to put the boundaries.

JSX Copy

```jsx
// Strategy 1: One boundary at the top (simple, but jarring transitions)
<Suspense fallback={<FullPageSpinner />}>
  <Routes>
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/settings" element={<Settings />} />
  </Routes>
</Suspense>

// Strategy 2: Per-route boundaries with skeleton screens (better UX)
<Routes>
  <Route path="/dashboard" element={
    <Suspense fallback={<DashboardSkeleton />}>
      <Dashboard />
    </Suspense>
  } />
  <Route path="/settings" element={
    <Suspense fallback={<SettingsSkeleton />}>
      <Settings />
    </Suspense>
  } />
</Routes>

// Strategy 3: Per-section within a page (best UX for complex pages)
function Dashboard() {
  return (
    <div>
```

```
      </Suspense>
      <Suspense fallback={<TableSkeleton />}>
        <LazyDataTable />  {/* Loads independently */}
      </Suspense>
    </div>
  );
}
```

Strategy 3 is what production apps use. Different sections of a page load independently, with their own skeleton loaders. The page feels fast because the lightweight parts appear instantly.

Common Mistakes

Mistake 1: Not analyzing your bundle. Most developers have no idea that 60% of their bundle is unused lodash or an enormous date library. Run an analyzer before optimizing anything else.

Mistake 2: Lazy loading small components. The overhead of creating a separate chunk and loading it over the network is worse than just including a 5KB component in the main bundle. Only lazy load page-level components or heavy libraries.

Mistake 3: Using CommonJS in a modern project. `require()` prevents tree shaking. Use `import` / `export` for everything, and choose dependencies that ship ES modules.

Interview Answer

> "Code splitting breaks the JavaScript bundle into smaller chunks that load on demand. I use React.lazy with dynamic import for route-level splitting, so users only download the code for pages they actually visit. For heavy third-party libraries like chart.js, I dynamically import them in event handlers or useEffect so

ES module syntax. For the loading experience, I place Suspense boundaries strategically with skeleton screens so different sections of a page load independently."

✅ Key Takeaways

- **Static import** bundles everything upfront. **Dynamic import()** creates separate chunks loaded on demand.
- **Bundle analysis** reveals what's actually large. Always analyze before optimizing.
- **Tree shaking** removes unused code. Requires ES module syntax (import/export, not require).
- Replace heavy libraries: moment.js with dayjs, lodash with lodash-es or specific imports.
- **Lazy load at the page/route level**, not small components.
- Place **Suspense boundaries** per-section for the best loading experience with skeleton screens.

QUICK CHECK — no marks, just for you.

1. According to the lesson, what is the key difference between a static `import` statement and a dynamic `import()` call?

- ○ A dynamic import() is a function call that returns a Promise, and the bundler creates a separate chunk that loads only when the import executes; a static import is resolved at build time and included in the main bundle.
- ○ A dynamic import() prevents the module from being tree shaken, while a static import allows tree shaking.

- [ ] Both produce the same bundle output; dynamic import() is only nicer syntax for use inside async functions.

Check answer

2. A project imports lodash with `const _ = require('lodash')` and uses only `_.debounce`. The bundle still contains all 72KB of lodash. Based on the lesson, why?

- [ ] lodash is listed in package-lock.json, and every locked dependency is always bundled in full regardless of syntax.
- [ ] Tree shaking only runs in development builds, so the unused lodash code is stripped later at runtime.
- [ ] Tree shaking removes imported-but-unused code, but it relies on ES module syntax; require() is CommonJS, so the bundler pulls in the entire library.
- [ ] Tree shaking only removes unused local source files, never anything inside node_modules.

Check answer

3. A developer wraps every component in the app — including a 5KB `<Badge />` used on most screens — in React.lazy with its own Suspense boundary. What does the lesson identify as the problem, and what is the recommended granularity?

- [ ] React.lazy can only be called once per application, so additional lazy components silently fall back to static imports.
- [ ] Suspense boundaries cannot be nested, so each extra lazy component throws at render time.

libraries instead.

It is correct and optimal — maximum splitting always yields the fastest initial load, and the lesson recommends it.

Check answer

Previous
React Performance: The Complete Toolkit

Next
Virtualization: Rendering 10,000 Rows

LEARN MORE

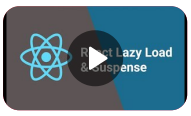
ARTICLES & DOCS

- lazy — API reference
  react.dev
- Suspense — API reference
  react.dev

VIDEOS



- React lazy loading explained (lazy + Suspense)
  YouTube



- Lazy loading & Suspense for performance
  YouTube



- Code splitting & Suspense
  YouTube

Hand-picked references for this lesson.